

# FISM 2026 - Lezione 03/06/2026

Esercizi pratici: siti web statici, CSS intermedio, Bootstrap, GitHub Pages e JSON opzionale

Questo PDF contiene solo le tracce degli esercizi. Le soluzioni sono nel file separato incluso nello ZIP.

## Indice degli esercizi

| N. | Sezione                   | Titolo                                      | Diff.      | Tempo  |
|----|---------------------------|---------------------------------------------|------------|--------|
| 01 | A. Repository e consegna  | Audit iniziale della repository             | Base       | 15 min |
| 02 | A. Repository e consegna  | Ricostruire una repository disordinata      | Base       | 20 min |
| 03 | A. Repository e consegna  | README minimo ma utile                      | Base       | 25 min |
| 04 | A. Repository e consegna  | CHANGELOG della giornata                    | Base       | 10 min |
| 05 | A. Repository e consegna  | Troubleshooting realistico                  | Intermedio | 20 min |
| 06 | B. Tema, HTML e contenuto | Scegliere un tema non documentale           | Base       | 15 min |
| 07 | B. Tema, HTML e contenuto | Homepage semantica generica                 | Base       | 25 min |
| 08 | B. Tema, HTML e contenuto | Correggere link e percorsi relativi         | Base       | 20 min |
| 09 | B. Tema, HTML e contenuto | Sito multipagina minimo                     | Intermedio | 30 min |
| 10 | B. Tema, HTML e contenuto | Immagini accessibili e ordinate             | Base       | 20 min |
| 11 | C. CSS intermedio         | Pagina leggibile con max-width              | Base       | 20 min |
| 12 | C. CSS intermedio         | Spaziature coerenti                         | Base       | 20 min |
| 13 | C. CSS intermedio         | Card con Flexbox                            | Base       | 25 min |
| 14 | C. CSS intermedio         | Responsive: card impilate su mobile         | Intermedio | 20 min |
| 15 | C. CSS intermedio         | Galleria con CSS Grid                       | Intermedio | 30 min |
| 16 | C. CSS intermedio         | Hero e call to action                       | Base       | 25 min |
| 17 | C. CSS intermedio         | Debug di navbar rotta                       | Intermedio | 20 min |
| 18 | C. CSS intermedio         | Revisione critica del design                | Intermedio | 25 min |
| 19 | D. Bootstrap ragionato    | Collegare Bootstrap nel modo corretto       | Base       | 15 min |
| 20 | D. Bootstrap ragionato    | Container e spacing utilities               | Base       | 20 min |
| 21 | D. Bootstrap ragionato    | Grid Bootstrap con card                     | Intermedio | 30 min |
| 22 | D. Bootstrap ragionato    | Colori semantici Bootstrap                  | Base       | 20 min |
| 23 | D. Bootstrap ragionato    | Content Bootstrap: testo, immagini, tabelle | Intermedio | 30 min |
| 24 | D. Bootstrap ragionato    | Navbar Bootstrap spiegata                   | Intermedio | 35 min |
| 25 | D. Bootstrap ragionato    | Accordion FAQ Bootstrap                     | Intermedio | 30 min |
| 26 | D. Bootstrap ragionato    | Refactoring con utilities                   | Avanzato   | 35 min |
| 27 | E. GitHub Pages           | Preparare la pubblicazione                  | Base       | 15 min |
| 28 | E. GitHub Pages           | Diagnosticare Pages che non funziona        | Intermedio | 25 min |

| N. | Sezione               | Titolo                               | Diff.      | Tempo     |
|----|-----------------------|--------------------------------------|------------|-----------|
| 29 | E. GitHub Pages       | Aggiornare README dopo pubblicazione | Base       | 10 min    |
| 30 | E. GitHub Pages       | Issue di pubblicazione               | Intermedio | 20 min    |
| 31 | F. JSON/API opzionale | Progettare data.json                 | Intermedio | 25 min    |
| 32 | F. JSON/API opzionale | Renderizzare card da data.json       | Avanzato   | 40 min    |
| 33 | F. JSON/API opzionale | Documentare dati/API                 | Intermedio | 25 min    |
| 34 | F. JSON/API opzionale | Fallback se il JSON non si carica    | Avanzato   | 30 min    |
| 35 | G. Sprint finale      | Sprint finale di consegna            | Finale     | 45-60 min |

# A. Repository e consegna

## Esercizio 01 - Audit iniziale della repository

Difficoltà: Base | Tempo stimato: 15 min

**Obiettivo.** Capire se la repository è pronta per diventare un progetto consegnabile.

### Consegna.

- 1 Apri la repository del progetto.
- 2 Controlla se esistono i file obbligatori.
- 3 Scrivi una lista di problemi da correggere.
- 4 Fai commit della checklist in README.md o in docs/checklist.md.

**File coinvolti.** README.md, docs/checklist.md opzionale

### Vincoli.

- Non correggere ancora i problemi: prima devi individuarli.
- La checklist deve essere leggibile da un compagno.

### Suggerimenti.

- Usa git status prima di iniziare.
- Controlla anche nomi file con maiuscole/minuscole.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 02 - Ricostruire una repository disordinata

Difficoltà: Base | Tempo stimato: 20 min

**Obiettivo.** Sistemare una struttura di progetto confusa.

### Situazione di partenza.

```
progetto/  
|-- Index.html  
|-- STYLE.CSS  
|-- readme.txt  
|-- immagini varie/  
|   |-- Screenshot finale.PNG  
|-- guida utente.docx
```

### Consegna.

- 1 Parti dalla struttura sbagliata mostrata sotto.
- 2 Rinomina i file in modo corretto.
- 3 Crea le cartelle mancanti.
- 4 Sposta immagini e documentazione nelle cartelle giuste.

**File coinvolti.** tutta la repository

### Vincoli.

- Usa nomi minuscoli per i file HTML/CSS.
- Evita spazi nei nomi delle immagini.

- Non cancellare contenuto senza motivo.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 03 - README minimo ma utile

*Difficoltà: Base | Tempo stimato: 25 min*

**Obiettivo.** Scrivere un README che orienti subito chi apre la repository.

### Consegna.

- 1 Crea o migliora README.md.
- 2 Inserisci descrizione, funzionalità, struttura, documentazione, sito online e licenza.
- 3 Lascia il link GitHub Pages come placeholder se non è ancora attivo.

**File coinvolti.** README.md

### Vincoli.

- Non scrivere un testo generico tipo "questo è il progetto".
- Ogni funzionalità deve avere una descrizione concreta.

### Suggerimenti.

- Il sito può essere generico: portfolio, catalogo, dashboard, evento, ecc.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 04 - CHANGELOG della giornata

*Difficoltà: Base | Tempo stimato: 10 min*

**Obiettivo.** Registrare le modifiche principali fatte durante la lezione.

### Consegna.

- 1 Crea o aggiorna CHANGELOG.md.
- 2 Aggiungi una versione 0.2.0 per il lavoro del 03/06.
- 3 Dividi le modifiche in Aggiunto, Modificato e Corretto.

**File coinvolti.** CHANGELOG.md

### Vincoli.

- Non scrivere solo "aggiornato progetto".
- Le voci devono essere verificabili nella repository.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 05 - Troubleshooting realistico

*Difficoltà: Intermedio | Tempo stimato: 20 min*

**Obiettivo.** Scrivere una pagina di troubleshooting utile per problemi veri del sito.

### Consegna.

- 1 Crea docs/troubleshooting.md.
- 2 Inserisci almeno 4 problemi comuni.
- 3 Per ogni problema scrivi causa possibile, controllo e soluzione.

**File coinvolti.** docs/troubleshooting.md

#### **Vincoli.**

- I problemi devono essere realistici per HTML/CSS/GitHub Pages.
- Non copiare frasi vaghe come "controllare tutto".

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## **B. Tema, HTML e contenuto**

### **Esercizio 06 - Scegliere un tema non documentale**

*Difficoltà: Base | Tempo stimato: 15 min*

**Obiettivo.** Definire un sito generico, non per forza legato alla documentazione.

#### **Consegna.**

- 1 Scegli un tema per il sito finale.
- 2 Compila una tabella con target, sezioni, immagini e dati.
- 3 Aggiorna README o docs/architettura.md.

**File coinvolti.** README.md oppure docs/architettura.md

#### **Vincoli.**

- Non scegliere "sito documentazione" se non è il tema che vuoi davvero.
- Il tema deve essere realizzabile in HTML/CSS statico.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

### **Esercizio 07 - Homepage semantica generica**

*Difficoltà: Base | Tempo stimato: 25 min*

**Obiettivo.** Costruire una struttura HTML sensata per un sito generico.

#### **Consegna.**

- 1 Crea una homepage per il tema scelto.
- 2 Usa header, nav, main, section e footer.
- 3 Inserisci almeno 4 sezioni reali.

**File coinvolti.** index.html

#### **Vincoli.**

- Non usare solo div.
- Ogni section deve avere un titolo h2.

- I link della navbar devono puntare alle sezioni.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 08 - Correggere link e percorsi relativi

*Difficoltà: Base | Tempo stimato: 20 min*

**Obiettivo.** Imparare a diagnosticare immagini e link rotti.

### Situazione di partenza.

```
<link rel="stylesheet" href="/css/style.css">

<a href="/docs/Guida Utente.md">Guida</a>
```

### Consegna.

- 1 Correggi i percorsi sbagliati nel frammento HTML.
- 2 Assumi che index.html sia nella root.
- 3 Assumi che le immagini siano in assets/immagini/.

**File coinvolti.** index.html

### Vincoli.

- Usa percorsi relativi.
- Evita spazi nei nomi file.
- Usa testo alternativo descrittivo.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 09 - Sito multipagina minimo

*Difficoltà: Intermedio | Tempo stimato: 30 min*

**Obiettivo.** Creare un sito con più pagine mantenendo una navigazione coerente.

### Consegna.

- 1 Crea index.html, catalogo.html e contatti.html.
- 2 Inserisci la stessa navbar in tutte le pagine.
- 3 Collega lo stesso style.css.
- 4 Aggiungi un link alla repository GitHub.

**File coinvolti.** index.html, catalogo.html, contatti.html, style.css

### Vincoli.

- Tutte le pagine devono essere raggiungibili dalla navbar.
- Non usare percorsi assoluti locali tipo C:\Users\...

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 10 - Immagini accessibili e ordinate

*Difficoltà: Base | Tempo stimato: 20 min*

**Obiettivo.** Usare immagini in modo ordinato e con alt text sensato.

### Consegna.

- 1 Inserisci 3 immagini nel sito.
- 2 Spostale in assets/immagini/.
- 3 Rinominale con nomi chiari.
- 4 Scrivi alt text descrittivi.

**File coinvolti.** index.html, assets/immagini/

### Vincoli.

- No nomi tipo Screenshot 2026-06-03 alle 12.31.png.
- No alt="immagine".

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## C. CSS intermedio

### Esercizio 11 - Pagina leggibile con max-width

*Difficoltà: Base | Tempo stimato: 20 min*

**Obiettivo.** Evitare pagine larghe e testo difficile da leggere.

### Consegna.

- 1 Aggiungi regole CSS globali.
- 2 Centra il contenuto principale.
- 3 Rendi le immagini responsive.

**File coinvolti.** style.css

### Vincoli.

- Non usare width fissa in pixel sul body.
- Il sito deve adattarsi a schermi piccoli.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

### Esercizio 12 - Spaziature coerenti

*Difficoltà: Base | Tempo stimato: 20 min*

**Obiettivo.** Eliminare l'effetto pagina amatoriale dovuto a spazi casuali.

### Consegna.

- 1 Definisci spaziature coerenti per header, section, card e footer.
- 2 Usa valori ricorrenti: 8, 16, 24, 32, 48, 64 px.
- 3 Evita numeri casuali.

**File coinvolti.** style.css

### Vincoli.

- Non usare margin diversi a caso per ogni elemento.
- Le sezioni devono essere chiaramente separate.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 13 - Card con Flexbox

*Difficoltà: Base | Tempo stimato: 25 min*

**Obiettivo.** Creare una riga di card ordinate e flessibili.

### Consegna.

- 1 Crea un contenitore .cards.
- 2 Inserisci almeno 3 .card.
- 3 Usa Flexbox per disporle in riga.

**File coinvolti.** index.html, style.css

### Vincoli.

- Le card devono avere uguale larghezza su desktop.
- Devono avere spazio tra loro.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 14 - Responsive: card impilate su mobile

*Difficoltà: Intermedio | Tempo stimato: 20 min*

**Obiettivo.** Rendere le card leggibili anche da telefono.

### Consegna.

- 1 Aggiungi una media query.
- 2 Sotto 768px, disponi navbar e card in colonna.
- 3 Verifica ridimensionando il browser.

**File coinvolti.** style.css

### Vincoli.

- Non creare due versioni diverse della pagina.
- Usa CSS responsive.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 15 - Galleria con CSS Grid

*Difficoltà: Intermedio | Tempo stimato: 30 min*

**Obiettivo.** Usare Grid per una galleria responsive di immagini o elementi.

### Consegna.



- 1 Crea una sezione galleria.
- 2 Usa CSS Grid con auto-fit o auto-fill.
- 3 Fai in modo che le colonne si adattino allo spazio disponibile.

**File coinvolti.** index.html, style.css

### Vincoli.

- Non usare tre colonne fisse se poi su mobile si rompe.
- Le immagini devono restare dentro le card.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 16 - Hero e call to action

*Difficoltà: Base | Tempo stimato: 25 min*

**Obiettivo.** Creare una sezione iniziale chiara e orientata all'azione.

### Consegna.

- 1 Crea una hero con titolo, sottotitolo e bottone.
- 2 Il bottone deve portare alla sezione principale.
- 3 La hero deve avere contrasto sufficiente.

**File coinvolti.** index.html, style.css

### Vincoli.

- Non usare un bottone senza destinazione.
- Il testo deve spiegare subito il sito.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 17 - Debug di navbar rotta

*Difficoltà: Intermedio | Tempo stimato: 20 min*

**Obiettivo.** Correggere una navbar con spaziature e link disordinati.

### Situazione di partenza.

```
.navbar a {  
  margin: 50px;  
  color: black;  
}  
  
.navbar {  
  background: gray;  
}
```

### Consegna.

- 1 Parti dal CSS sbagliato.
- 2 Rendi la navbar orizzontale su desktop.
- 3 Rendila verticale su mobile.
- 4 Rimuovi sottolineatura dai link.

**File coinvolti.** style.css

### Vincoli.

- Non usare margin enorme sui link.
- Usa gap sul contenitore.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 18 - Revisione critica del design

*Difficoltà: Intermedio | Tempo stimato: 25 min*

**Obiettivo.** Saper valutare se un sito statico è presentabile.

### Consegna.

- 1 Scambia il sito con un compagno.
- 2 Compila una review tecnica.
- 3 Indica almeno 3 problemi e 3 miglioramenti concreti.

**File coinvolti.** docs/review.md oppure issue GitHub

### Vincoli.

- La review deve essere specifica.
- Non basta scrivere "bello" o "brutto".

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## D. Bootstrap ragionato

### Esercizio 19 - Collegare Bootstrap nel modo corretto

*Difficoltà: Base | Tempo stimato: 15 min*

**Obiettivo.** Capire ordine e ruolo dei file Bootstrap e CSS personalizzato.

### Consegna.

- 1 Aggiungi Bootstrap via CDN.
- 2 Aggiungi il tuo style.css dopo Bootstrap.
- 3 Spiega in un commento perché style.css va dopo.

**File coinvolti.** index.html

### Vincoli.

- Non rimuovere il tuo CSS.
- Non inserire il JS Bootstrap dentro head se non serve.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

### Esercizio 20 - Container e spacing utilities

*Difficoltà: Base | Tempo stimato: 20 min*

**Obiettivo.** Usare Bootstrap non a caso, ma per controllare larghezza e spaziature.

### Consegna.

- 1 Avvolgi il contenuto principale in .container.
- 2 Usa my-5, mb-4, p-4 dove ha senso.
- 3 Annota il significato di almeno 3 classi usate.

**File coinvolti.** index.html

### Vincoli.

- Non mettere classi Bootstrap casuali.
- Ogni classe utility scelta deve avere uno scopo.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 21 - Grid Bootstrap con card

*Difficoltà: Intermedio | Tempo stimato: 30 min*

**Obiettivo.** Costruire una sezione a card responsive usando la griglia.

### Consegna.

- 1 Crea una sezione con 3 card.
- 2 Usa row, g-4 e col-md-4.
- 3 Ogni card deve avere titolo, testo e bottone.

**File coinvolti.** index.html

### Vincoli.

- Non usare tabelle per fare layout.
- Le card devono impilarsi automaticamente su mobile.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 22 - Colori semantici Bootstrap

*Difficoltà: Base | Tempo stimato: 20 min*

**Obiettivo.** Capire che primary, success, warning e danger non sono solo colori, ma significati.

### Consegna.

- 1 Crea quattro bottoni o alert con colori semantici.
- 2 Scrivi accanto a ciascuno quando usarlo.
- 3 Evita colori scelti solo per gusto personale.

**File coinvolti.** index.html

### Vincoli.

- Usa colori Bootstrap coerenti con il messaggio.
- Non usare danger per una informazione neutra.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 23 - Content Bootstrap: testo, immagini, tabelle

*Difficoltà: Intermedio | Tempo stimato: 30 min*

**Obiettivo.** Usare Bootstrap anche per contenuto, non solo componenti vistosi.

### Consegna.

- 1 Crea una sezione con titolo, paragrafo lead, immagine responsive e tabella.
- 2 Usa classi Bootstrap per rendere leggibile il contenuto.
- 3 La tabella deve essere responsive.

**File coinvolti.** index.html

### Vincoli.

- Non lasciare immagini enormi.
- La tabella deve stare dentro table-responsive.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 24 - Navbar Bootstrap spiegata

*Difficoltà: Intermedio | Tempo stimato: 35 min*

**Obiettivo.** Usare una navbar responsive capendo le parti principali.

### Consegna.

- 1 Inserisci una navbar Bootstrap.
- 2 Modifica brand e link.
- 3 Fai funzionare il menu su schermi piccoli.
- 4 Aggiungi il JS bundle Bootstrap se necessario.

**File coinvolti.** index.html

### Vincoli.

- Il valore di data-bs-target deve corrispondere all'id del menu.
- La navbar deve puntare a sezioni esistenti.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 25 - Accordion FAQ Bootstrap

*Difficoltà: Intermedio | Tempo stimato: 30 min*

**Obiettivo.** Creare una FAQ interattiva con Bootstrap senza scrivere JS custom.

### Consegna.

- 1 Crea una sezione FAQ con accordion.
- 2 Inserisci almeno 3 domande.

- 3 Usa testi coerenti con il tema del sito.

**File coinvolti.** index.html

### Vincoli.

- Ogni elemento deve avere id univoco.
- data-bs-parent deve puntare all'id dell'accordion.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 26 - Refactoring con utilities

*Difficolta: Avanzato | Tempo stimato: 35 min*

**Obiettivo.** Sostituire CSS ripetitivo con utility Bootstrap leggibili.

### Situazione di partenza.

```
<section class="box-special">
  <h2>Offerta speciale</h2>
  <p>Testo introduttivo.</p>
</section>
```

```
.box-special {
  padding: 24px;
  margin-top: 48px;
  margin-bottom: 48px;
  background: #f8f9fa;
  border-radius: 8px;
}
```

### Consegna.

- 1 Parti dal frammento con classi custom.
- 2 Riscrivilo usando utilities Bootstrap dove ha senso.
- 3 Mantieni style.css solo per personalizzazioni vere.

**File coinvolti.** index.html, style.css

### Vincoli.

- Non trasformare tutto in una zuppa illeggibile di classi.
- Usa utilities solo quando rendono il codice piu chiaro.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## E. GitHub Pages

### Esercizio 27 - Preparare la pubblicazione

*Difficolta: Base | Tempo stimato: 15 min*

**Obiettivo.** Verificare che il sito sia pubblicabile prima di attivare Pages.

### Consegna.

- 1 Controlla che index.html sia nella root.
- 2 Controlla CSS, immagini e link.

- 3 Fai commit e push.
- 4 Scrivi nel README una sezione Sito online con placeholder.

**File coinvolti.** README.md, index.html, style.css

### Vincoli.

- Non attivare Pages se la repo è ancora sporca.
- Non lasciare modifiche non committate.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 28 - Diagnosticare Pages che non funziona

*Difficoltà: Intermedio | Tempo stimato: 25 min*

**Obiettivo.** Riconoscere gli errori più comuni nella pubblicazione.

### Situazione di partenza.

Sintomi:

1. GitHub Pages mostra 404.
2. Il sito si apre ma senza CSS.
3. Le immagini si vedono in locale ma non online.
4. Il sito online non mostra l'ultima modifica.

### Consegna.

- 1 Leggi i sintomi.
- 2 Individua la causa più probabile.
- 3 Scrivi la correzione in docs/troubleshooting.md.

**File coinvolti.** docs/troubleshooting.md

### Vincoli.

- Per ogni sintomo scrivi una causa e una soluzione concreta.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 29 - Aggiornare README dopo pubblicazione

*Difficoltà: Base | Tempo stimato: 10 min*

**Obiettivo.** Rendere il link pubblico facilmente trovabile.

### Consegna.

- 1 Aggiungi link GitHub Pages al README.
- 2 Aggiungi anche link alla repository e alla documentazione.
- 3 Fai commit.

**File coinvolti.** README.md

### Vincoli.

- Il link deve essere cliccabile.
- Non mettere il link solo nella descrizione della repository.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## Esercizio 30 - Issue di pubblicazione

Difficoltà: Intermedio | Tempo stimato: 20 min

**Obiettivo.** Documentare un problema tecnico come issue GitHub.

### Consegna.

- 1 Apri una issue per un problema GitHub Pages.
- 2 Usa titolo chiaro, descrizione, passi per riprodurre, risultato atteso e soluzione proposta.
- 3 Chiudi la issue con una pull request o un commit.

**File coinvolti.** GitHub Issues

### Vincoli.

- Non scrivere solo "non funziona".
- La issue deve aiutare a correggere il problema.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## F. JSON/API opzionale

### Esercizio 31 - Progettare data.json

Difficoltà: Intermedio | Tempo stimato: 25 min

**Obiettivo.** Separare dati e HTML per un sito statico più ordinato.

### Consegna.

- 1 Scegli una lista di dati coerente con il tema.
- 2 Crea data.json con almeno 5 elementi.
- 3 Ogni elemento deve avere almeno 3 campi.

**File coinvolti.** data.json

### Vincoli.

- Il JSON deve essere valido.
- Usa virgolette doppie.
- Non mettere commenti dentro JSON.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

### Esercizio 32 - Renderizzare card da data.json

Difficoltà: Avanzato | Tempo stimato: 40 min

**Obiettivo.** Usare fetch per caricare dati locali e mostrarli nella pagina.

### Consegna.

- 1 Crea un contenitore nel file HTML.

- 2 Collega script.js.
- 3 Carica data.json con fetch.
- 4 Crea una card per ogni elemento.

**File coinvolti.** index.html, script.js, data.json

#### **Vincoli.**

- Questo e extra: chi non ha finito CSS/Pages deve prima finire quelli.
- Il sito deve essere testato su GitHub Pages o con server locale.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## **Esercizio 33 - Documentare dati/API**

*Difficoltà: Intermedio | Tempo stimato: 25 min*

**Obiettivo.** Spiegare struttura e uso dei dati nel progetto.

#### **Consegna.**

- 1 Crea docs/api.md.
- 2 Documenta data.json o API usata.
- 3 Inserisci tabella dei campi ed esempio JSON.

**File coinvolti.** docs/api.md

#### **Vincoli.**

- Se usi una API esterna, indica endpoint e fonte.
- Se usi data.json, documenta comunque il formato.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.

## **Esercizio 34 - Fallback se il JSON non si carica**

*Difficoltà: Avanzato | Tempo stimato: 30 min*

**Obiettivo.** Gestire l'errore di caricamento dati in modo visibile all'utente.

#### **Consegna.**

- 1 Modifica script.js.
- 2 Se fetch fallisce, mostra un messaggio nella pagina.
- 3 Non limitarti a console.error.

**File coinvolti.** script.js

#### **Vincoli.**

- Il messaggio deve apparire nel DOM.
- Usa un alert Bootstrap o una classe CSS simile.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.



# G. Sprint finale

## Esercizio 35 - Sprint finale di consegna

*Difficoltà: Finale | Tempo stimato: 45-60 min*

**Obiettivo.** Portare il progetto a uno stato consegnabile.

### Consegna.

- 1 Scegli 5 correzioni prioritarie.
- 2 Lavora solo su quelle.
- 3 Aggiorna README e CHANGELOG.
- 4 Pubblica su GitHub Pages.
- 5 Fai commit finale.

**File coinvolti.** tutta la repository

### Vincoli.

- Non iniziare funzionalità nuove se il sito non è pubblicato.
- Non lasciare la repo sporca.
- Il link Pages deve funzionare.

**Output atteso.** Modifiche salvate nella repository e commit coerente con il lavoro svolto.